

Delving into Large Language Models for Effective Time-Series Anomaly Detection

주요 문제

LLM을 Time-Series Anomaly Detection(TSAD)에 적용하면 단순한 baseline보다도 성능이 낮은 경우가 많았다

- text로 시계열 입력 → 정상 변동도 anomaly로 오탐 (false positive 과다)
- image로 시계열 입력 → 눈에 띄는 spike만 탐지, subtle anomaly는 놓침

기존 연구들은 "성능이 낮다"만 보고했지 "왜 LLM이 TSAD를 못하는가"를 분석하지 않음

문제 분해

- 1) Understanding — 이 시계열에 anomaly가 있는가/없는가 (binary classification)
- 2) Localization — anomaly가 정확히 어느 index 구간인가

Understanding 실패 분석

localization 없이 "anomaly 존재 여부만" 판단하는 instance-level detection으로 단순화해서 실험.

결과:

- Point anomaly, Range anomaly → image prompt 사용 시 거의 완벽 탐지
- Trend anomaly, Frequency anomaly → 항상 같은 label을 출력하는 constant classifier baseline과 비슷하거나 더 낮음

CoT, few-shot 등 NLP에서 효과적인 전략을 다 써봐도 trend/frequency anomaly는 개선이 안 됨

원인: trend와 seasonality가 뒤섞여 있어서 LLM이 반복적 seasonal pattern에 가려진 subtle anomaly를 인식 못함

Localization 실패 분석

반대로, anomaly 구간을 image에 파란색으로 직접 표시. localization만 테스트하는 설정.

결과:

- 모든 모델에서 성능이 올라가긴 하지만 여전히 제한적
- frequency anomaly처럼 산발적으로 흩어진 구간이 많을수록 localization 실패율 증가
- anomaly 구간 수가 많고 각 구간이 짧을수록 오류 증가

원인: 기존 prompt는 값만 나열 (0.12 0.15 -0.03 0.88 ...). 하지만 LLM은 token counting 능력이 매우 취약함

제안 방법

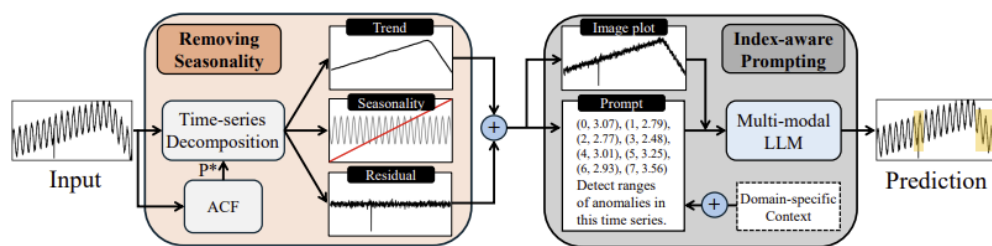


Figure 4: An overview of our proposed method to mitigate understanding and localization challenges.

1) De-seasonalization: Understanding 문제 해결

LLM이 seasonal pattern을 anomaly로 오인하거나, seasonality에 가려진 anomaly를 놓치는 문제를 해결. LLM에 입력 전에 seasonal component만 통계적으로 제거

seasonality만 빼는 이유: seasonality는 ACF로 안정적으로 추정 가능하지만, trend나 residual을 잘못 제거하면 anomaly 자체를 소실시킬 위험이 있기 때문

3단계 pipeline:

1. ACF로 dominant period P^* 탐지
2. classical additive decomposition으로 $x_t = \text{seasonality} + \text{trend} + \text{residual}$ 분리
3. de-seasonalized sequence $\tilde{x}_t = x_t - s_t$ 를 LLM에 전달

연산 비용: FFT 기반 $O(T \log T)$, LLM inference 비용 대비 무시 가능

2) Index-aware Prompting: Localization 문제 해결

index-aware: (0, 0.12), (1, 0.15), (2, -0.03), (3, 0.88), ... → anomaly 값 발견하면 앞의 index를 읽기만 하면 됨

counting 문제를 쉬운 lookup 문제로 변환. input token은 ~2배 증가하지만 localization accuracy가 크게 향상.

실험 결과

- **AnomLLM Benchmark**

21가지 기존 전략 전부 상회, 평균 39.9% 최대 66.6% F1 향상. GPT-4o 기준 기존 text only F1 17.76은 naive baseline(31.75)보다도 낮았으나, 제안 방법은 text only(54.40)만으로 기존 image 최고 성능(42.03)을 넘김. text+image 결합 시 70.04. 기존 affiliation metric의 localization 둔감성을 지적하고 standard F1을 병행 사용

→ "text는 image보다 열등하다"는 기존 통념을 뒤집음. index를 붙인 text가 x축 눈금이 부정확한 image보다 localization에 유리

- **모델별 차이**

MMLU-Pro(일반 reasoning)가 높을수록 향상 폭이 큼: GPT-4o(74.68) +28, Qwen(71.59) +21, Gemini(64.09) +20, InternVL2(56.20) +1. 기초 추론이 약하면 prompting 개선도 무의미

→ MMLU-Pro를 model 선택의 lightweight proxy로 활용 가능

- **Ablation**

각 component 제거 시 F1 하락: index -23.36(가장 치명적), value -16.88, image -14.24, de-seasonalization -5.64.

→ index가 가장 핵심, text와 image는 complementary

- **LLM 자체 decomposition**

component 존재 판단은 통계 방법과 비슷하게 가능하나, 수열 직접 생성은 중간부터 오류 급증하며 실패

→ tool 사용 여부 결정은 LLM이, 실제 계산은 외부 tool이 담당하는 구조가 적합

- **Index-free vs Index-aware**

GT 제공 상태에서 index 유무만 비교: Gemini F1 27→63(2배), Qwen F1 20→60(3배). index 없으면 Qwen이 스스로 Python code를 생성하는 기현상 발생, output token 573개로 폭증(있을 때 59개)

→ input token이 ~2배 늘어나는 비용 대비 localization accuracy 향상이 압도적, counting 문제를 lookup으로

- **Non-LLM 비교**

16개 ML/DL/FM 베이스라인과 TSB-AD-U에서 비교. LLM 기반 방법이 평균 F1 최고 (0.34)를 달성했으나, 추론 속도는 시리즈당 ~140초로 기존 방법(수초 이내) 대비 느림.

→ 정확도 vs 속도 trade-off 존재. 오프라인/저자원 환경에 적합.

실용적 가치 논의

TSB-AD-U benchmark에서 ML/DL/Foundation Model 총 16개 방법과 비교. accuracy(F1)는 제안 방법이 최고이나, speed는 ~140초로 가장 느림(SR은 0.02초). LLM은 anomaly 구간을 text로 token 단위 decoding해야 하기 때문

→ real-time에는 부적합, zero-shot/offline 환경에서 유리

LLM 고유 강점: context 기반 anomaly filtering

기존 방법: 숫자 패턴이 비정상이면 무조건 anomaly로 판단

현업에서는 예정된 전력 제한, 월말 유지보수 등 설명 가능한 pattern이 false positive으로 잡히는 경우가 빈번함

LLM: domain context를 natural language로 prompt에 추가하는 것만으로 해당 구간을 제외 가능

→ 기존 연구는 "이런 게 anomaly다" 방향, 본 논문은 "이런 건 anomaly가 아니다" 반대 방향으로 알려진 event를 exclude해서 unknown anomaly에 집중.

결론

LLM 기반 TSAD의 실패 원인을 Understanding과 Localization으로 분리 진단하고, de-seasonalization + index-aware prompting이라는 해결책으로 기존 21개 prompting 전략 대비 최대 66.6% F1 향상 달성. LLM의 한계(속도)와 고유 강점(context-based filtering) 제시